第 2 章 • 预备知识 / 2.1

2.1 数据操作

用形状、运算和存储关系，把一组数值组织成正确的计算。

创建与形状 → 运算与广播 → 索引与修改 → 共享与转换

本节目标

读完后，能够判断张量形状和广播结果，按样本与特征取数据，区分共享存储与独立复制，并发现损失计算中的形状错配。

建议先连续阅读完整知识链，再运行示例，最后集中完成章末练习。已会的内容可快速通过，重点追踪自己无法解释的结果。

教材定位：PDF 第 58-65 页 / 正文第 40-47 页。

参考《动手学深度学习》Release 2.0.0（2023-08-18）及本项目已验证讲义整理。此 PDF 为独立学习读本，配套程序与练习均在项目内。

统一的数据

本节统一使用三条样本、每条两个特征：

PYTHON

```
import torch

data = [[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]]
X = torch.tensor(data, dtype=torch.float32)
```

每行代表样本、每列代表特征，是本例的约定。张量本身只存数值和结构，不自动知道样本的含义。

2.1.1 创建与形状

张量是多维数组。单个数、向量、矩阵、带更多轴的数组，都可以用张量表示。PyTorch 张量支持 GPU 运算，也能在记录相关计算后求梯度。

写法	当前含义
`X.shape`	每个轴的长度，本例为 `[3, 2]`
`X.ndim`	轴的数量，本例为 2
`X.numel()`	元素总数，本例为 6
`X.dtype`	元素的数据类型，本例为 float32
`X.device`	存储位置，本例为 CPU

两个轴不等于两个样本；本例是二维张量，有三条样本、每条两个特征。

常见创建方法：

```python
torch.arange(1, 7)  # [1, 2, 3, 4, 5, 6]，包含起点、不含终点
torch.zeros(3, 2)   # 三行两列的全 0
torch.ones(3, 2)    # 三行两列的全 1
torch.randn(3, 2)   # 从标准正态分布采样的随机数
torch.zeros_like(X) # 与 X 对齐形状、dtype、device 的全 0
```

整数 arange 默认产生整数张量；可指定 `dtype=torch.float32` 创建浮点数。随机数常用于初始化参数；本节演示固定随机种子，方便复查结果，概率原理在 2.6 讲。

变形与转置：

```python
X.reshape(2, 3)   # [[1,2,3], [4,5,6]]
X.reshape(-1, 3)  # 已知每行 3 个元素，自动推断为 2 行
X.T               # [[1,3,5], [2,4,6]]，二维转置
```

对当前连续存储的数据，reshape 按元素顺序重新组织；二维转置交换两个轴。它们可以形状相同，内容排列却不同。

目标形状的元素总数必须匹配；一次 reshape 最多指定一个 -1。六个元素无法 reshape 成四行两列。

调用 reshape 返回结果，原变量不会自动改形状。reshape 可能返回共享存储的视图，也可能复制，不能把它当作独立复制；需要独立数据时用 clone。

原三条样本每条两个特征，reshape 为两行三列会重新划分哪些元素落在同一行。数据的样本和特征语义需要结合任务检查。

2.1.2 运算、比较、汇总与拼接

设 `Y = torch.ones_like(X) * 2`，则 X 和 Y 的形状均为 `[3, 2]`。

`+`、`-`、`*`、`/`、`**` 对对应位置的元素运算：`X*Y` 是逐个位置相乘，`X**2` 是每个数平方。矩阵乘法使用 `@`，尺寸和推导在 2.3 正式展开。

`torch.exp(X)` 对每个元素计算以 e 为底的指数。`X==Y`、`X<Y`、`X>Y` 返回同形状的布尔张量，各位置为 True 或 False。

```python
X.sum()                     # 全部元素相加，21
X.sum(dim=0)                # 每列相加，[9,12]，形状 (2,)
X.sum(dim=1)                # 每行相加，[3,7,11]，形状 (3,)
X.sum(dim=1, keepdim=True)  # 每行相加，保留形状 (3,1)
X.mean(dim=0)               # 每列的均值，[3,4]
```

dim 指操作的轴。沿轴 0 汇总，是把各行相同列的数合起来，剩下每列的结果；沿轴 1 汇总，是把每行的各列合起来，剩下每行的结果。通常汇总的轴会消失，keepdim=True 将它保留为长度 1。

```python
torch.cat((X, Y), dim=0)  # 沿行拼接，形状 (6,2)
torch.cat((X, Y), dim=1)  # 沿列拼接，形状 (3,4)
```

cat 将已有数据端对端拼接。沿哪个轴拼接，哪个轴的长度增加；其他轴的长度必须匹配。增加样本通常沿行拼接，增加特征通常沿列拼接，这仍取决于数据约定。

2.1.3 广播：不同形状怎样计算

广播允许兼容的不同形状执行按元素运算。规则是：

1. 从最右边的轴向左对齐。
2. 对应轴长度相同，或者其中一个为 1，就兼容。
3. 缺失的轴当作长度 1，将这些长度 1 的轴扩展到对方长度。

可以想象为重复使用某些元素来匹配形状，一般无需手工复制输入；计算结果仍需存储。

两个输入形状	结果
`(3,2)` 与标量	`(3,2)`，标量用于每个位置
`(3,2)` 与 `(2,)`	`(3,2)`，两个值分别用于每列
`(3,1)` 与 `(1,2)`	`(3,2)`
`(2,1,3)` 与 `(1,4,1)`	`(2,4,3)`
`(2,3)` 与 `(2,)`	不兼容，末轴 3 与 2 不相同，也没有 1

综合演示故意执行并捕获最后一项错误，以说明规则。看到“预期的广播错误”属于示例内容，后续会正常继续。

一个有实际意义的操作是每个特征减去自身均值：

```python
feature_mean = X.mean(dim=0)  # (2,)，每列均值
centered = X - feature_mean  # (3,2) - (2,)，广播到每条样本
```

结果每列均值为 0，不需要用 Python 循环逐行减。

2.1.4 索引、切片、筛选与赋值

索引从 0 开始，负索引从后往前。切片包含起点、不包含终点。

```python
X[0]        # 第一行
X[-1]       # 最后一行
X[1:3]      # 第 2、3 行
X[1,0]     # 第 2 行、第 1 列
X[:2,:]     # 前两行，所有列
X[:,1]      # 第 2 列，形状 (3,)
X[:,1:2]    # 第 2 列，但保留二维，形状 (3,1)
X[X > 3]    # 条件筛选，得到一维的 [4,5,6]
```

`X>3` 是与原矩阵同形状的布尔条件；`X[X>3]` 是把满足条件的元素取出来。这两种结果的含义和形状不同。

`X[:,1]` 与 `X[:,1:2]` 的数值看起来相同，但前者使用整数索引去掉一个轴，后者使用切片保留这个轴；后面的广播行为会不同。

写入数据：

```python
modified = X.clone()
modified[1,0] = 99
modified[:2,:] = 12
```

一项赋值改一个位置，切片赋值改一整片。这里先独立复制，所以原 X 保持不变。

2.1.5 内存：变量、视图和复制

变量可以指向同一个张量对象；不同张量对象也可能共享底层存储。先区分重新绑定与原地修改：

```python
Y = Y + X  # 计算新结果，再让名字 Y 指向它
Y += X     # 本节条件下，在原来的张量上更新
Z[:] = X + Y  # 保留目标 Z，把结果写入它
```

切片赋值保留目标张量，右侧表达式仍可能产生临时结果。它不意味着整个运算没有任何临时内存。

`id()` 查看 Python 对象的身份，不能用它直接判断所有张量是否共享同一底层数据。

```python
a = torch.tensor([1.0, 2.0, 3.0])
alias = a          # 同一个对象
view = a[1:]       # 基本切片，共享原数据
snapshot = a.clone()  # 独立复制
view[0] = 99
```

修改后 a 与 alias 都能看到 `[1,99,3]`，snapshot 仍是 `[1,2,3]`。不是所有索引都会共享数据，这里说的是本例的基本切片。

本节使用不追踪梯度的张量。前面的梯度参数更新继续使用 torch.no_grad；记录梯度后的原地操作限制在自动微分部分讨论。

2.1.6 NumPy 与 Python 数值

转换要区分共享和复制：

写法	当前普通 CPU 张量/数组的行为
`array = tensor.numpy()`	数组和张量共享底层数据
`torch.from_numpy(array)`	张量与数组共享底层数据
`torch.tensor(array)`	复制数据，创建独立张量
`tensor.numpy().copy()`	创建独立 NumPy 数组
`tensor.clone()`	创建独立张量

教材对转换后的共享内存有概括，但不能将其理解成“所有转换都共享”。实测 torch.tensor(array) 会复制；from_numpy(array) 共享。

GPU 张量需要先放到 CPU；追踪梯度的张量通常先 detach。常见写法为 `tensor.detach().cpu().numpy()`。detach 不表示独立复制；如果需要独立数组，最后再加 `.copy()`。

只有一个元素的张量可以取成 Python 数值：

```python
a = torch.tensor([3.5])
a.item()  # 3.5
float(a)  # 3.5
int(a)    # 3, 转为整数
```

将整节连接到一个实际计算

给每条样本增加一个“两个特征的总和”特征：

```python
row_total = X.sum(dim=1, keepdim=True)  # (3,1)
augmented = torch.cat((X, row_total), dim=1)  # (3,3)
```

结果为 `[[1,2,3],[3,4,7],[5,6,11]]`。这是创建、汇总、保留轴与拼接在同一任务中的组合。

再连接前面学过的损失：

```python
prediction = X[:,1:2]  # (3,1)
target = X[:,1]        # (3,)
wrong_error = prediction - target       # (3,3)，出现样本之间的交叉比较
correct_error = prediction - target.reshape(-1,1)  # (3,1)，逐样本对齐
```

两组数值明明相同，错误形状导致 MSE 约 5.3333，对齐后 MSE 为 0。广播能提高效率，也需要根据任务核对是否正在比较对应的样本。

综合练习：把操作放在同一个任务中

使用本讲义开头的 X（三条样本、每条两个特征），集中完成以下操作。先预测形状，再写表达式和运行。

任务	目标与约束
1. 重新组织	将 X 变为两行三列
2. 转置	交换二维 X 的行与列
3. 按列中心化	每个元素减去所在列的均值，保留 (3,2)
4. 每行总和	将两个特征求和，保留 (3,1)
5. 添加新特征	将每行总和拼到 X 后，得到 (3,3)
6. 索引与切片	取最后两条样本的全部特征
7. 条件比较	标记 X 中大于 3 的位置，保留布尔矩阵
8. 三维广播	计算下面 a+b，解释结果形状

PYTHON

```
a = torch.arange(6).reshape(2, 1, 3)
b = torch.arange(4).reshape(1, 4, 1)
```

集中解释三个问题：

1. reshape 与转置形状相同，为什么元素排列不同？
2. (3,1) 的预测减去 (3,) 的标签会发生什么？如何逐样本对齐？
3. alias、基本切片、clone、from_numpy 与 torch.tensor(array)，哪些共享原数据，哪些独立？用修改一个元素的实验说明。

配套练习文件：`exercises/ch02/01_data_operations.py`。记录操作结果、预期与实际的差异，以及自己的解释；无需背诵全部 API。

阅读后的速查表

需要做什么	首先想到的工具
检查结构	shape、ndim、numel、dtype、device
调整排列	reshape；二维交换轴用 .T
批量计算	按元素运算和广播
汇总或增加数据	sum/mean；cat
取样本或特征	索引、切片、布尔筛选
保留独立副本	clone；NumPy 数组用 copy
导出普通对象	numpy、from_numpy、item

配套完整演示：`lessons/ch02/01_data_operations.py`。程序包含故意捕获的广播错误，用来说明不兼容形状；它不是需要修复的运行失败。

资料说明：教材作者为 Aston Zhang、Zachary C. Lipton、Mu Li、Alexander J. Smola；页码基于用户提供的中文 PDF。文中相对路径均从“李沐-深度学习”项目根目录计算。已验证环境：Python 3.8.20、PyTorch 2.4.1+cu121、NumPy 1.23.5、pandas 2.0.3。整理日期：2026-10-15。